

Roll No.: C014	Name: N Mehul
Class : B.Tech CE	Batch :2023-2027
Date of Experiment:	Date of Submission
Grade :	

AIM:

- Determine interest points using SIFT parameters. Use interest points to match corner points of two images.
- Rotate and scale one image and match corner points
- Comment on the results before and after changing the parameters

Algorithm:

Step 1: Load and Preprocess Images

- Reads an image from file.
- IMREAD_GRAYSCALE converts image to grayscale (needed for feature extraction).

Step 2: Create SIFT Detector & Extract Features

Function:

```
cv2.SIFT_create (nfeatures=0, contrastThreshold=0.04,
edgeThreshold=10, sigma=1.6)
detectAndCompute(image, mask)
```

Example:

```
sift = cv2.SIFT_create (nfeatures=500, contrastThreshold=0.04)
keypoints, descriptors = sift.detectAndCompute(img, None)
```

Explanation:

- `nfeatures`: maximum number of keypoints to detect.
 - `contrastThreshold`: filter out weak features (higher → fewer, stronger points).
 - `sigma`: Gaussian blur before detection (affects scale).
 - `detectAndCompute` returns:
 - **keypoints** (interest points with position, scale, orientation).
 - **descriptors** (feature vectors used for matching).
-

Step 3: Match Features Between Two Images

Function:

```
cv2.BFMatcher()
```

```
knnMatch(queryDescriptors, trainDescriptors, k=2)
```

Example:

```
bf = cv2.BFMatcher()
```

```
matches = bf.knnMatch(des1, des2, k=2)
```

Explanation:

- `BFMatcher`: brute-force matcher compares each descriptor in one image to all in the other.
- `knnMatch`: finds **k best matches** (commonly `k=2`).

- Needed for **Lowe's ratio test**.
-

Step 4: Apply Lowe's Ratio Test

Logic:

for m, n in matches:

 if m.distance < ratio * n.distance:

 good_matches.append(m)

Example:

good = []

for m, n in matches:

 if m.distance < 0.75 * n.distance:

 good.append(m)

Explanation:

- Keeps only matches where the best match is **significantly better** than the second-best.
 - Reduces false matches.
-

Step 5: Draw Matches

Function:

```
cv2.drawMatches(img1, kp1, img2, kp2, matches, outImg, flags)
```

Example:

```
result = cv2.drawMatches(img1, kp1, img2, kp2, good, None,  
  
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

Explanation:

- Draws lines between matched keypoints.
- NOT_DRAW_SINGLE_POINTS avoids drawing unmatched keypoints.

CODE:

```
import cv2  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
# Step 1: Load Images in Grayscale  
  
img1 = cv2.imread('Building.jpg', cv2.IMREAD_GRAYSCALE)  
img2 = cv2.imread('Building.jpg', cv2.IMREAD_GRAYSCALE)  
  
# Check if images are loaded  
  
if img1 is None or img2 is None:  
    print("Error loading images")  
    exit()  
  
# Step 2: Rotate and Scale Second Image
```

```
height, width = img2.shape

# Rotation matrix (rotate 30 degrees)
rotation_matrix = cv2.getRotationMatrix2D((width/2, height/2), 30, 1)

# Rotate image
img2_rotated = cv2.warpAffine(img2, rotation_matrix, (width, height))

# Scale image
img2_scaled = cv2.resize(img2_rotated, None, fx=1.2, fy=1.2)

# Step 3: Create SIFT Detector
sift = cv2.SIFT_create(nfeatures=500, contrastThreshold=0.04)

# Detect keypoints and descriptors
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2_scaled, None)

# Step 4: Feature Matching using BFMatcher
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

# Step 5: Apply Lowe's Ratio Test
good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

# Step 6: Draw Matches
result = cv2.drawMatches(
    img1, kp1,
```

```

img2_scaled, kp2,
good_matches, None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Step 7: Display Results
plt.figure(figsize=(12,6))
plt.imshow(result)
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()

# Print Information
print("Number of Keypoints in Image 1:", len(kp1))
print("Number of Keypoints in Image 2:", len(kp2))
print("Number of Good Matches:", len(good_matches))

# Step 8: Change Parameters and Compare
print("\nChanging SIFT parameters...")
sift2 = cv2.SIFT_create(nfeatures=800, contrastThreshold=0.08)
kp1_new, des1_new = sift2.detectAndCompute(img1, None)
kp2_new, des2_new = sift2.detectAndCompute(img2_scaled, None)
matches2 = bf.knnMatch(des1_new, des2_new, k=2)
good_matches2 = []
for m, n in matches2:
    if m.distance < 0.75 * n.distance:

```

```
        good_matches2.append(m)
result2 = cv2.drawMatches(
    img1, kp1_new,
    img2_scaled, kp2_new,
    good_matches2, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.figure(figsize=(12,6))
plt.imshow(result2)
plt.title("SIFT Matching After Parameter Change")
plt.axis("off")
plt.show()
print("Keypoints after parameter change (Image 1):", len(kp1_new))
print("Keypoints after parameter change (Image 2):", len(kp2_new))
print("Good matches after parameter change:", len(good_matches2))
```

OUTPUT:

SIFT Feature Matching



```
Number of Keypoints in Image 1: 500  
Number of Keypoints in Image 2: 500  
Number of Good Matches: 261
```

Conclusion:

In this experiment, the SIFT algorithm was used to detect important points in images and match them between two images. The algorithm successfully found keypoints and matched similar features using a feature matcher and Lowe's ratio test. Even after rotating and scaling one image, many keypoints were still matched correctly, which shows that SIFT works well with changes in rotation and size. When the SIFT parameters were changed, the number of detected keypoints and matches also changed. Overall, the experiment shows that SIFT is a reliable method for detecting and matching features between images.